

Chapitre 3 — Base de Données PostgreSQL

3.1 Schéma Prisma (source de vérité)

Le schéma Prisma définit l'intégralité des tables, relations, index et contraintes.

```
// prisma/schema.prisma

generator client {
  provider      = "prisma-client-js"
  previewFeatures = ["fullTextSearch"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

3.2 Enumérations

```
enum UserRole {
  buyer
  seller
  admin
  delivery
}

enum UserStatus {
  active
  inactive
  suspended
  banned
}

enum OrderStatus {
  pending
  confirmed
  processing
  shipped
  delivered
  cancelled
  refunded
}

enum PaymentStatus {
  pending
  processing
  completed
}
```

```
    failed
    refunded
    partially_refunded
}

enum PaymentMethod {
    credit_card
    debit_card
    paypal
    apple_pay
    google_pay
    stripe
    mobile_money
    bank_transfer
    cash_on_delivery
}

enum ProductStatus {
    draft
    active
    inactive
    archived
    suspended
}

enum SellerPlan {
    basic
    pro
    enterprise
}

enum TicketStatus {
    open
    in_progress
    waiting
    resolved
    closed
}

enum TicketPriority {
    low
    medium
    high
    urgent
}

enum NotificationType {
    order
    payment
    shipping
    promotion
    system
}
```

```
support
}
```

3.3 Tables

3.3.1 Utilisateurs

```
model User {
  id          String      @id @default(uuid()) @db.Uuid
  email       String      @unique @db.VarChar(255)
  name        String      @db.VarChar(255)
  passwordHash String     @map("password_hash") @db.VarChar(255)
  phone       String?     @db.VarChar(20)
  avatar      String?     @db.VarChar(500)
  role        UserRole    @default(buyer)
  status      UserStatus  @default(active)
  emailVerified Boolean    @default(false) @map("email_verified")
  lastLoginAt DateTime?   @map("last_login_at")
  createdAt   DateTime     @default(now()) @map("created_at")
  updatedAt   DateTime     @updatedAt @map("updated_at")

  // Relations
  addresses      Address[]
  seller          Seller?
  orders         Order[]      @relation("BuyerOrders")
  cart           Cart?
  wishlist       WishlistItem[]
  reviews        Review[]
  supportTickets SupportTicket[]
  notifications  Notification[]
  refreshTokens  RefreshToken[]

  @@map("users")
}

model RefreshToken {
  id          String      @id @default(uuid()) @db.Uuid
  token       String      @unique @db.Text
  userId      String      @map("user_id") @db.Uuid
  expiresAt   DateTime     @map("expires_at")
  createdAt   DateTime     @default(now()) @map("created_at")

  user        User        @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@map("refresh_tokens")
}
```

3.3.2 Adresses

```

model Address {
  id          String    @id @default(uuid()) @db.Uuid
  userId      String    @map("user_id") @db.Uuid
  label       String    @db.VarChar(100)    // "Domicile", "Bureau", etc.
  firstName   String    @map("first_name") @db.VarChar(100)
  lastName    String    @map("last_name") @db.VarChar(100)
  address1    String    @db.VarChar(255)
  address2    String?   @db.VarChar(255)
  city        String    @db.VarChar(100)
  state       String?   @db.VarChar(100)
  postalCode  String    @map("postal_code") @db.VarChar(20)
  country     String    @db.VarChar(100)
  phone       String?   @db.VarChar(20)
  isDefault   Boolean   @default(false) @map("is_default")
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")

  user        User      @relation(fields: [userId], references: [id], onDelete:
Cascade)
  orders      Order[]

  @@index([userId])
  @@map("addresses")
}

```

3.3.3 Vendeurs (Seller Profile)

```

model Seller {
  id          String    @id @default(uuid()) @db.Uuid
  userId      String    @unique @map("user_id") @db.Uuid
  shopName    String    @map("shop_name") @db.VarChar(255)
  slug        String    @unique @db.VarChar(255)
  description  String?   @db.Text
  logo        String?   @db.VarChar(500)
  banner      String?   @db.VarChar(500)
  category    String    @db.VarChar(100)
  country     String    @db.VarChar(100)
  businessType String    @default("individual") @map("business_type")
  @db.VarChar(50)
  plan        SellerPlan @default(basic)
  commissionRate Decimal @default(0.12) @map("commission_rate")
  @db.Decimal(5, 4)
  rating      Decimal    @default(0) @db.Decimal(3, 2)
  totalSales  Int        @default(0) @map("total_sales")
  totalRevenue Decimal    @default(0) @map("total_revenue") @db.Decimal(12, 2)
  verified    Boolean    @default(false)
  active      Boolean    @default(true)
  bankAccountJson Json?  @map("bank_account") // { iban, bic, holderName }
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")
}

```

```

    user          User          @relation(fields: [userId], references: [id],
onDelete: Cascade)
    products       Product[]
    orders         OrderItem[] @relation("SellerOrders")
    payouts        Payout[]

    @@index([userId])
    @@index([slug])
    @@index([category])
    @@map("sellers")
}

```

3.3.4 Catégories

```

model Category {
  id          String      @id @default(uuid()) @db.Uuid
  name        String      @db.VarChar(100)
  slug        String      @unique @db.VarChar(100)
  description String?     @db.Text
  icon        String?     @db.VarChar(50)
  image       String?     @db.VarChar(500)
  parentId    String?     @map("parent_id") @db.Uuid
  sortOrder   Int         @default(0) @map("sort_order")
  active       Boolean    @default(true)
  createdAt   DateTime    @default(now()) @map("created_at")
  updatedAt   DateTime    @updatedAt @map("updated_at")

  parent      Category? @relation("CategoryTree", fields: [parentId], references:
[id])
  children    Category[] @relation("CategoryTree")
  products    Product[]

  @@index([parentId])
  @@index([slug])
  @@map("categories")
}

```

3.3.5 Produits

```

model Product {
  id          String      @id @default(uuid()) @db.Uuid
  sellerId    String      @map("seller_id") @db.Uuid
  categoryId  String      @map("category_id") @db.Uuid
  name        String      @db.VarChar(255)
  slug        String      @unique @db.VarChar(255)
  description  String      @db.Text
  shortDescription String? @map("short_description") @db.VarChar(500)
  price       Decimal     @db.Decimal(10, 2)
}

```

```

compareAtPrice  Decimal?    @map("compare_at_price") @db.Decimal(10, 2)
costPrice       Decimal?    @map("cost_price") @db.Decimal(10, 2)
sku             String?     @unique @db.VarChar(100)
barcode         String?     @db.VarChar(100)
stock           Int         @default(0)
lowStockThreshold Int      @default(5) @map("low_stock_threshold")
weight          Decimal?    @db.Decimal(8, 2) // en grammes
dimensions      Json?       // { length, width, height }
status          ProductStatus @default(draft)
featured        Boolean     @default(false)
trending        Boolean     @default(false)
averageRating   Decimal     @default(0) @map("average_rating") @db.Decimal(3,
2)
reviewCount     Int         @default(0) @map("review_count")
salesCount      Int         @default(0) @map("sales_count")
viewCount       Int         @default(0) @map("view_count")
tags            String[]    @db.VarChar(50)
metaTitle       String?     @map("meta_title") @db.VarChar(255)
metaDescription String?     @map("meta_description") @db.VarChar(500)
publishedAt     DateTime?   @map("published_at")
createdAt       DateTime    @default(now()) @map("created_at")
updatedAt       DateTime    @updatedAt @map("updated_at")

seller          Seller       @relation(fields: [sellerId], references: [id])
category        Category     @relation(fields: [categoryId], references: [id])
images          ProductImage[]
variants        ProductVariant[]
orderItems      OrderItem[]
reviews         Review[]
wishlistItems   WishlistItem[]
cartItems       CartItem[]

@@index([sellerId])
@@index([categoryId])
@@index([status])
@@index([featured])
@@index([createdAt])
@@index([price])
@@index([name])
@@map("products")
}

model ProductImage {
  id          String @id @default(uuid()) @db.Uuid
  productId   String @map("product_id") @db.Uuid
  url         String @db.VarChar(500)
  alt         String? @db.VarChar(255)
  sortOrder   Int    @default(0) @map("sort_order")
  isPrimary   Boolean @default(false) @map("is_primary")
  createdAt   DateTime @default(now()) @map("created_at")

  product     Product @relation(fields: [productId], references: [id], onDelete:
Cascade)

```

```

    @@index([productId])
    @@map("product_images")
}

model ProductVariant {
  id          String  @id @default(uuid()) @db.Uuid
  productId   String  @map("product_id") @db.Uuid
  name        String  @db.VarChar(100)    // "Rouge / XL"
  sku         String? @unique @db.VarChar(100)
  price       Decimal @db.Decimal(10, 2)
  stock       Int     @default(0)
  attributes  Json?   // { color: "rouge", size: "XL" }
  createdAt   DateTime @default(now()) @map("created_at")

  product     Product @relation(fields: [productId], references: [id], onDelete:
Cascade)
  cartItems   CartItem[]
  orderItems  OrderItem[]

  @@index([productId])
  @@map("product_variants")
}

```

3.3.6 Panier

```

model Cart {
  id          String  @id @default(uuid()) @db.Uuid
  userId      String  @unique @map("user_id") @db.Uuid
  couponId    String? @map("coupon_id") @db.Uuid
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

  user        User    @relation(fields: [userId], references: [id], onDelete:
Cascade)
  coupon      Coupon? @relation(fields: [couponId], references: [id])
  items       CartItem[]

  @@map("carts")
}

model CartItem {
  id          String  @id @default(uuid()) @db.Uuid
  cartId      String  @map("cart_id") @db.Uuid
  productId   String  @map("product_id") @db.Uuid
  variantId   String? @map("variant_id") @db.Uuid
  quantity    Int     @default(1)
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

  cart        Cart    @relation(fields: [cartId], references: [id],
onDelete: Cascade)
}

```

```

product    Product          @relation(fields: [productId], references: [id])
variant    ProductVariant? @relation(fields: [variantId], references: [id])

@@unique([cartId, productId, variantId])
@@map("cart_items")
}

```

3.3.7 Coupons

```

model Coupon {
  id          String    @id @default(uuid()) @db.Uuid
  code        String    @unique @db.VarChar(50)
  description String?   @db.VarChar(255)
  discountType String    @map("discount_type") @db.VarChar(20) // "percentage" |
"fixed"
  discountValue Decimal @map("discount_value") @db.Decimal(10, 2)
  minAmount    Decimal? @map("min_amount") @db.Decimal(10, 2)
  maxDiscount  Decimal? @map("max_discount") @db.Decimal(10, 2)
  usageLimit   Int?     @map("usage_limit")
  usedCount    Int      @default(0) @map("used_count")
  startDate    DateTime @map("start_date")
  endDate      DateTime @map("end_date")
  active       Boolean  @default(true)
  createdAt    DateTime @default(now()) @map("created_at")

  carts        Cart[]
  orders        Order[]

  @@index([code])
  @@map("coupons")
}

```

3.3.8 Commandes

```

model Order {
  id          String    @id @default(uuid()) @db.Uuid
  orderNumber String    @unique @map("order_number") @db.VarChar(50)
  buyerId    String    @map("buyer_id") @db.Uuid
  addressId   String    @map("address_id") @db.Uuid
  couponId    String?   @map("coupon_id") @db.Uuid
  status      OrderStatus @default(pending)
  subtotal    Decimal    @db.Decimal(10, 2)
  shippingCost Decimal    @default(0) @map("shipping_cost") @db.Decimal(10,
2)
  taxAmount    Decimal    @default(0) @map("tax_amount") @db.Decimal(10, 2)
  discountAmount Decimal    @default(0) @map("discount_amount")
@db.Decimal(10, 2)
  totalAmount  Decimal    @map("total_amount") @db.Decimal(10, 2)
  currency     String     @default("EUR") @db.VarChar(3)
}

```



```

    notes                String?      @db.Text
    shippingMethod        String?      @map("shipping_method") @db.VarChar(50)
    trackingNumber         String?      @map("tracking_number") @db.VarChar(100)
    shippedAt              DateTime?    @map("shipped_at")
    deliveredAt            DateTime?    @map("delivered_at")
    cancelledAt            DateTime?    @map("cancelled_at")
    cancelReason           String?      @map("cancel_reason") @db.Text
    createdAt              DateTime     @default(now()) @map("created_at")
    updatedAt              DateTime     @updatedAt @map("updated_at")

    buyer                 User          @relation("BuyerOrders", fields: [buyerId],
references: [id])
    address               Address       @relation(fields: [addressId], references: [id])
    coupon                Coupon?      @relation(fields: [couponId], references: [id])
    items                 OrderItem[]
    payments              Payment[]
    shipments             Shipment[]

    @@index([buyerId])
    @@index([status])
    @@index([createdAt])
    @@index([orderId])
    @@map("orders")
}

model OrderItem {
    id                String            @id @default(uuid()) @db.Uuid
    orderId           String            @map("order_id") @db.Uuid
    productId         String            @map("product_id") @db.Uuid
    variantId         String?           @map("variant_id") @db.Uuid
    sellerId          String            @map("seller_id") @db.Uuid
    productName       String            @map("product_name") @db.VarChar(255)
    productImage      String?           @map("product_image") @db.VarChar(500)
    quantity          Int
    unitPrice         Decimal            @map("unit_price") @db.Decimal(10, 2)
    totalPrice        Decimal            @map("total_price") @db.Decimal(10, 2)
    commission        Decimal            @default(0) @db.Decimal(10, 2)
    status            OrderStatus       @default(pending)
    createdAt          DateTime          @default(now()) @map("created_at")

    order             Order             @relation(fields: [orderId], references: [id],
onDelete: Cascade)
    product           Product           @relation(fields: [productId], references: [id])
    variant           ProductVariant?   @relation(fields: [variantId], references: [id])
    seller            Seller            @relation("SellerOrders", fields: [sellerId],
references: [id])

    @@index([orderId])
    @@index([sellerId])
    @@index([productId])
    @@map("order_items")
}

```

3.3.9 Paiements

```

model Payment {
  id                String      @id @default(uuid()) @db.Uuid
  orderId           String      @map("order_id") @db.Uuid
  amount            Decimal     @db.Decimal(10, 2)
  currency          String      @default("EUR") @db.VarChar(3)
  method            PaymentMethod
  status            PaymentStatus @default(pending)
  stripePaymentId   String?     @map("stripe_payment_id") @db.VarChar(255)
  paypalOrderId     String?     @map("paypal_order_id") @db.VarChar(255)
  mobileMoneyRef    String?     @map("mobile_money_ref") @db.VarChar(255)
  refundAmount      Decimal     @default(0) @map("refund_amount")
  @db.Decimal(10, 2)
  refundReason      String?     @map("refund_reason") @db.Text
  metadata          Json?
  paidAt            DateTime?   @map("paid_at")
  createdAt         DateTime    @default(now()) @map("created_at")
  updatedAt         DateTime    @updatedAt @map("updated_at")

  order             Order       @relation(fields: [orderId], references: [id])

  @@index([orderId])
  @@index([stripePaymentId])
  @@index([status])
  @@map("payments")
}

```

3.3.10 Expéditions

```

model Shipment {
  id                String      @id @default(uuid()) @db.Uuid
  orderId           String      @map("order_id") @db.Uuid
  carrier           String      @db.VarChar(100) // "DHL", "FedEx", "La Poste",
etc.
  trackingNumber    String?     @map("tracking_number") @db.VarChar(255)
  trackingUrl       String?     @map("tracking_url") @db.VarChar(500)
  status            String      @default("preparing") @db.VarChar(50)
  estimatedDelivery DateTime? @map("estimated_delivery")
  shippedAt         DateTime? @map("shipped_at")
  deliveredAt       DateTime? @map("delivered_at")
  weight            Decimal?    @db.Decimal(8, 2)
  cost              Decimal?    @db.Decimal(10, 2)
  createdAt         DateTime    @default(now()) @map("created_at")
  updatedAt         DateTime    @updatedAt @map("updated_at")

  order             Order       @relation(fields: [orderId], references: [id])

  @@index([orderId])
  @@index([trackingNumber])
}

```

```

    @@map("shipments")
}

```

3.3.11 Virements vendeurs (Payouts)

```

model Payout {
  id          String    @id @default(uuid()) @db.Uuid
  sellerId    String    @map("seller_id") @db.Uuid
  amount      Decimal   @db.Decimal(10, 2)
  currency    String    @default("EUR") @db.VarChar(3)
  status      String    @default("pending") @db.VarChar(20) // pending, processing,
completed, failed
  method      String    @db.VarChar(50)    // bank_transfer, paypal
  reference   String?   @db.VarChar(255)
  notes       String?   @db.Text
  paidAt      DateTime? @map("paid_at")
  createdAt   DateTime  @default(now()) @map("created_at")

  seller      Seller    @relation(fields: [sellerId], references: [id])

  @@index([sellerId])
  @@index([status])
  @@map("payouts")
}

```

3.3.12 Avis (Reviews)

```

model Review {
  id          String    @id @default(uuid()) @db.Uuid
  productId   String    @map("product_id") @db.Uuid
  userId      String    @map("user_id") @db.Uuid
  rating      Int       @db.Int // 1-5
  title       String?   @db.VarChar(255)
  comment     String?   @db.Text
  images      String[]  @db.VarChar(500)
  helpful     Int       @default(0)
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")

  product     Product   @relation(fields: [productId], references: [id], onDelete:
Cascade)
  user        User      @relation(fields: [userId], references: [id])

  @@unique([productId, userId])
  @@index([productId])
  @@index([userId])
  @@map("reviews")
}

```

3.3.13 Wishlist

```
model WishlistItem {
  id          String    @id @default(uuid()) @db.Uuid
  userId      String    @map("user_id") @db.Uuid
  productId   String    @map("product_id") @db.Uuid
  createdAt   DateTime  @default(now()) @map("created_at")

  user        User      @relation(fields: [userId], references: [id], onDelete:
Cascade)
  product     Product   @relation(fields: [productId], references: [id], onDelete:
Cascade)

  @@unique([userId, productId])
  @@map("wishlist_items")
}
```

3.3.14 Support (Tickets)

```
model SupportTicket {
  id          String    @id @default(uuid()) @db.Uuid
  userId      String    @map("user_id") @db.Uuid
  subject     String    @db.VarChar(255)
  category    String    @db.VarChar(50) // "order", "product", "account",
"technical", "other"
  priority    TicketPriority @default(medium)
  status      TicketStatus @default(open)
  orderId     String?    @map("order_id") @db.Uuid
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")

  user        User      @relation(fields: [userId], references: [id])
  messages    TicketMessage[]

  @@index([userId])
  @@index([status])
  @@map("support_tickets")
}

model TicketMessage {
  id          String    @id @default(uuid()) @db.Uuid
  ticketId    String    @map("ticket_id") @db.Uuid
  senderId    String    @map("sender_id") @db.Uuid
  message     String    @db.Text
  attachments String[]  @db.VarChar(500)
  isInternal  Boolean    @default(false) @map("is_internal")
  createdAt   DateTime  @default(now()) @map("created_at")

  ticket      SupportTicket @relation(fields: [ticketId], references: [id],
onDelete: Cascade)
```

```

    sender      User      @relation(fields: [senderId], references: [id])

    @@index([ticketId])
    @@map("ticket_messages")
}

```

3.3.15 Notifications

```

model Notification {
  id          String          @id @default(uuid()) @db.Uuid
  userId      String          @map("user_id") @db.Uuid
  type        NotificationType
  title       String          @db.VarChar(255)
  message     String          @db.Text
  data        Json?           // données contextuelles (orderId, etc.)
  read        Boolean         @default(false)
  createdAt   DateTime        @default(now()) @map("created_at")

  user        User            @relation(fields: [userId], references: [id],
onDelete: Cascade)

  @@index([userId, read])
  @@index([createdAt])
  @@map("notifications")
}

```

3.3.16 Paramètres système

```

model SystemSetting {
  id          String          @id @default(uuid()) @db.Uuid
  key         String          @unique @db.VarChar(100)
  value       Json
  updatedAt   DateTime        @updatedAt @map("updated_at")

  @@map("system_settings")
}

```

3.3.17 Fichiers uploadés

```

model UploadedFile {
  id          String          @id @default(uuid()) @db.Uuid
  filename     String          @db.VarChar(255)
  originalName String          @map("original_name") @db.VarChar(255)
  mimetype     String          @db.VarChar(100)
  size         Int             // en octets
  path         String          @db.VarChar(500)
}

```

```

uploaderId String? @map("uploader_id") @db.Uuid
createdAt DateTime @default(now()) @map("created_at")

@@index([uploaderId])
@@map("uploaded_files")
}

```

3.4 Index de performance

```

-- Recherche produits par texte (GIN index)
CREATE INDEX idx_products_search ON products
  USING GIN (to_tsvector('french', name || ' ' || COALESCE(description, '')));

-- Recherche fuzzy sur les noms de produits
CREATE INDEX idx_products_name_trgm ON products
  USING GIN (name gin_trgm_ops);

-- Recherche par fourchette de prix
CREATE INDEX idx_products_price ON products (price) WHERE status = 'active';

-- Commandes par statut et date (dashboard admin)
CREATE INDEX idx_orders_status_date ON orders (status, created_at DESC);

-- Notifications non lues par utilisateur
CREATE INDEX idx_notifications_unread ON notifications (user_id, created_at DESC)
  WHERE read = false;

```

3.5 Données de seed

Catégories par défaut

```

[
  { "name": "Électronique", "slug": "electronique", "icon": "📺" },
  { "name": "Mode & Vêtements", "slug": "mode-vetements", "icon": "👕" },
  { "name": "Maison & Jardin", "slug": "maison-jardin", "icon": "🏠" },
  { "name": "Sports & Loisirs", "slug": "sports-loisirs", "icon": "🏀" },
  { "name": "Beauté & Santé", "slug": "beaute-sante", "icon": "💄" },
  { "name": "Jouets & Enfants", "slug": "jouets-enfants", "icon": "🧸" },
  { "name": "Automobile", "slug": "automobile", "icon": "🚗" },
  { "name": "Livres & Médias", "slug": "livres-medias", "icon": "📖" },
  { "name": "Alimentation", "slug": "alimentation", "icon": "🍎" },
  { "name": "Art & Artisanat", "slug": "art-artisanat", "icon": "🎨" }
]

```

Admin par défaut

```
{  
  "email": "admin@globalmarket.com",  
  "name": "Administrateur",  
  "password": "Admin@GlobalMarket2026!",  
  "role": "admin"  
}
```